

TWO COMPUTATIONALLY CERTIFIED INDUCED-SATURATED GRAPHS FOR C_{12} AND C_{14}

ROMEO MUGNIER DE ALMEIDA

ABSTRACT. We give computer-assisted proofs for two explicit induced-saturated graphs. The graph specified by a released 182-vertex edge list and isomorphic to the Foster Census graph $\text{CAT}(182, 3)$ is C_{12} -induced-saturated, and the Coxeter graph in its Fano-plane model is C_{14} -induced-saturated. An elementary certificate lemma reduces both claims to finite statements about cycles with unique chords and induced paths with prescribed endpoints. Machine-readable certificates give one witness for every edge and nonedge, while separately written exhaustive programs recompute the relevant cycle and path data. For the Coxeter graph, a further C++ program examines all $\binom{28}{14} = 40,116,600$ vertex subsets to check the negative condition. These computations are not proof-assistant formalizations. We make no claim of priority or minimality.

1. INTRODUCTION

For a fixed graph H , an H -induced-saturated graph contains no induced copy of H , while deleting any present edge and adding any missing edge each creates an induced copy of H . Martin and Smith introduced an induced-saturation number in a generalized-graph framework [10]. Behrens, Erbes, Santana, Yager, and Yeager later studied the zero case, which yields ordinary H -induced-saturated graphs [2]; further existence results were obtained by Axenovich and Csikós [1].

Even cycles provide a particularly concrete existence problem. Fan, Hajebi, Hajebi, and Spirkl construct induced-saturated graphs for the even cycles through C_{10} , and deletion-saturated graphs for every even cycle [7]. Their subsequent work records that the existence of induced-saturated graphs for all even cycles remains open [8]. The present preliminary note records two finite constructions for C_{12} and C_{14} .

Theorem A. *The graph defined by the released 182-vertex edge list, which is isomorphic to $\text{CAT}(182, 3)$, is C_{12} -induced-saturated.*

Theorem B. *The Coxeter graph is C_{14} -induced-saturated.*

Both proofs use the same reduction. For an edge deletion we exhibit a cycle in which the deleted edge is the unique chord. For a nonedge addition we exhibit an induced path on the required number of vertices whose endpoints are the added pair. The graph-theoretic claims are thereby reduced to finite, explicitly enumerated statements.

This preliminary note does not establish priority for either construction. Its purpose is to record explicit, independently checkable computational certificates. No claim of minimality is made.

2. DEFINITIONS AND THE CERTIFICATE LEMMA

All graphs are finite, simple, and undirected. For a graph G , write $V(G)$ and $E(G)$ for its vertex and edge sets. If $e \in E(G)$, then $G - e$ denotes deletion of e . If $uv \notin E(G)$ with $u \neq v$, then $G + uv$ denotes addition of the edge uv . We write C_k for the cycle on k vertices and P_k for the path on k vertices.

Definition 2.1. Let H be a graph. A graph G is H -induced-saturated if

- (i) G contains no induced copy of H ;
- (ii) for every $e \in E(G)$, the graph $G - e$ contains an induced copy of H ; and
- (iii) for every $uv \notin E(G)$, the graph $G + uv$ contains an induced copy of H .

A *chord* of a specified cycle is an edge joining two nonconsecutive vertices of that cycle.

Date: Preliminary version, 25 July 2026.

Key words and phrases. induced saturation, even cycle, Coxeter graph, Foster Census, computational certificate.

Lemma 2.2 (Certificate lemma). *Let G be a graph and let $k \geq 4$.*

- (a) *If every k -cycle in G has at least one chord, then G contains no induced C_k .*
- (b) *If every edge $e \in E(G)$ is the unique chord of some k -cycle, then $G - e$ contains an induced C_k .*
- (c) *If every nonedge uv is the endpoint pair of an induced path on k vertices, then $G + uv$ contains an induced C_k .*

Proof. An induced k -cycle is precisely a k -cycle with no chord, which proves (a). For (b), let Q be a k -cycle whose only chord is e . After deleting e , the subgraph induced by $V(Q)$ consists exactly of the cycle edges of Q . For (c), an induced path on k vertices has $k - 1$ edges and no further edges among its vertices. Adding the edge between its endpoints therefore creates an induced k -cycle. $\square$

Remark 2.3. Stored-certificate validation and independent recomputation are distinct. The former checks that each listed cycle or path is a valid witness and that the witness keys give exact toggle coverage. The latter enumerates the relevant objects directly from the graph, without using the stored witness chosen for a particular toggle.

3. A C_{12} -INDUCED-SATURATED GRAPH

3.1. Construction and graph identity. Let G_{12} be the released edge-list graph. Its vertex set is $\{0, 1, \dots, 181\}$, and its edges are listed in `CAT182_3.edgelist`. The strict verifier reconstructs this graph and obtains

$$|V(G_{12})| = 182, \quad |E(G_{12})| = 273, \quad \binom{182}{2} - 273 = 16,198$$

nonedges. It also checks that every vertex has degree 3 and that the graph is connected.

Graph identity is checked separately from induced saturation. The certificate contains an LCF shift sequence whose reconstructed edge set is exactly the released edge-list graph. The direct GraphSym description accessed on 25 July 2026 lists the same shift sequence for CAT(182, 3) [4]. An alternative Foster Census LCF description included in the bundle is textually different; a separate NetworkX `GraphMatcher` computation reconstructs both graphs and returns an isomorphism with 182 mapping entries. Thus the claim made here is that the edge-list graph is *isomorphic* to CAT(182, 3); no equality of differently labelled census representations is asserted. For census context and the current catalogue, see [3, 5].

3.2. Cycles and edge deletions.

Proposition 3.1. *The graph G_{12} has exactly 273 simple 12-cycles. Every such cycle has exactly one chord, and every edge of G_{12} is the unique chord of exactly one such cycle.*

The primary program exhaustively extends simple 11-edge paths on 12 distinct vertices and closes them when the last vertex is adjacent to the initial vertex. It fixes the least cycle vertex as the start and breaks reversal symmetry by comparing the second and last vertices. For every retained cyclic order it subtracts the 12 ring edges from the full induced edge set. The result is the following complete chord distribution.

TABLE 1. Chord distribution of the simple 12-cycles in G_{12} .

Number of chords	Number of 12-cycles
1	273

A separately written standard-library verifier instead generates oriented simple 12-cycles from every start and canonicalizes only after closure by rotation and reversal. It reproduces the 273-cycle count, the chord histogram, and multiplicity one on every edge. It also performs a direct incremental chord-rejecting search in the original graph and finds no induced C_{12} . For every edge e , a further direct search in $G_{12} - e$, without reading the stored witness for e , finds an induced C_{12} .

The certificate separately stores one cyclic order for every edge. Its validator checks distinct and in-range vertices, all ring edges, the absence of unintended induced edges, and that the indexed edge is the unique chord. All 273 stored deletion witnesses pass.

3.3. Induced paths and nonedge additions.

Proposition 3.2. *The graph G_{12} has exactly 218,946 induced paths on 12 vertices, counted up to reversal. The set of endpoint pairs of these paths is precisely the set of all 16,198 nonedges of G_{12} .*

The exhaustive path search starts at every vertex and extends through every unused neighbour of the current endpoint. A new endpoint is accepted exactly when it has no neighbour among the earlier path vertices other than the current endpoint. This condition is necessary and sufficient to preserve inducedness. At 12 vertices, the program retains the orientation whose initial endpoint has smaller label. Both implementations obtain 218,946 paths and exact coverage of the 16,198 nonedges. This second computation proves every addition case without using the stored addition witnesses; it is an exhaustive induced-path endpoint computation, not a separate generic induced-cycle search in each augmented graph.

The certificate stores one induced 12-vertex path for each nonedge. For every entry the strict validator checks 12 distinct vertices, the 11 consecutive path edges, the absence of any other induced edge, and the exact endpoint pair. It then checks the toggled induced subgraph. All 16,198 stored addition witnesses pass.

Proof of Theorem A. By Proposition 3.1, every 12-cycle in G_{12} has a chord, so Lemma 2.2(a) implies that G_{12} contains no induced C_{12} . The same proposition states that every edge is the unique chord of a 12-cycle, so Lemma 2.2(b) implies that every edge deletion creates an induced C_{12} . Finally, Proposition 3.2 supplies an induced path on 12 vertices between the endpoints of every nonedge. Hence Lemma 2.2(c) implies that every nonedge addition creates an induced C_{12} . $\square$

4. THE COXETER GRAPH AND C_{14}

4.1. Fano-plane construction. Work in $\mathbb{Z}_7 = \{0, 1, \dots, 6\}$, with arithmetic modulo 7, and let

$$\mathcal{L} = \{\{i, i+1, i+3\} : i \in \mathbb{Z}_7\}.$$

These are the seven lines of a Fano plane. Define G_{14} by

$$(1) \quad V(G_{14}) = \binom{\mathbb{Z}_7}{3} \setminus \mathcal{L}, \quad XY \in E(G_{14}) \iff X \cap Y = \emptyset.$$

This is the standard nonline-triples model of the Coxeter graph [6, 9].

There are $\binom{7}{3} - 7 = 28$ vertices. We include the cubicity argument. Fix a nonlinear triple X . Each of its three pairs belongs to a distinct Fano line meeting X in two points; these three lines account for six of the nine point-line incidences involving the points of X . The remaining three incidences occur on three further lines, each meeting X in one point. Consequently exactly six Fano lines meet X , and exactly one Fano line is disjoint from X . The four triples contained in the four-point complement $\mathbb{Z}_7 \setminus X$ are precisely the possible disjoint triples. Exactly one is the disjoint Fano line, so the other three are graph neighbours of X . Thus G_{14} is cubic, has 42 edges, and has $\binom{28}{2} - 42 = 336$ nonedges.

The released Python verifiers reconstruct G_{14} from Equation (1). They check exact agreement with the 28-entry lexicographic triple-label map, the 42-edge file, and the label map embedded in the certificate. The primary verifier also recomputes connectedness, girth 7, diameter 4, and the unordered-pair distance distribution

$$d_1 : 42, \quad d_2 : 84, \quad d_3 : 168, \quad d_4 : 84.$$

The released programs do not compare this graph with an external Coxeter graph file; the identification as the Coxeter graph comes from the standard Fano-plane model cited above.

4.2. Cycles and edge deletions.

Proposition 4.1. *The Coxeter graph has exactly 420 simple 14-cycles. Of these, 252 have one chord and 168 have two chords; none has zero chords. Every edge is the unique chord of exactly six 14-cycles.*

TABLE 2. Chord distribution of the simple 14-cycles in the Coxeter graph.

Number of chords	Number of 14-cycles
0	0
1	252
2	168

The primary verifier uses a minimum-rooted depth-first search and breaks reversal symmetry. A separately written verifier uses integer adjacency masks, forms cycles by pairing internally vertex-disjoint 7-edge paths with common endpoints, and canonicalizes under every rotation and reversal. Both exhaustive methods give the distribution in Table 2. The 252 one-chord cycles distribute uniformly: each of the 42 edges is the unique chord six times.

The certificate contains one cyclic order for every edge. Both Python programs validate all 42 entries as unique-chord witnesses; the primary program also deletes the indexed chord and tests that the selected induced subgraph is connected and 2-regular.

4.3. Induced paths and nonedge additions.

Proposition 4.2. *The Coxeter graph has exactly 5,040 induced paths on 14 vertices, counted up to reversal. Every nonedge is the endpoint pair of at least one such path. The exact endpoint multiplicities are given in Table 3.*

TABLE 3. Endpoint multiplicities for induced 14-vertex paths.

Endpoint distance	Number of nonedges	Paths per pair
2	84	4
3	168	18
4	84	20

Both complete path searches use the same mathematical extension criterion as in Proposition 3.2, but one stores adjacency sets and the other integer masks. They agree on exact nonedge coverage and on

$$84 \cdot 4 + 168 \cdot 18 + 84 \cdot 20 = 5,040.$$

The certificate separately stores one path for every nonedge. Both Python programs validate the exact endpoints, 14 distinct vertices, the 13 path edges, and the absence of any other edge on the selected vertices. The primary program also adds the indexed nonedge and checks connected 2-regularity. All 336 addition witnesses pass.

Proof of Theorem B. By Proposition 4.1, every 14-cycle in the Coxeter graph has a chord. Thus Lemma 2.2(a) implies that the graph contains no induced C_{14} . The same proposition states that every edge is the unique chord of a 14-cycle, so Lemma 2.2(b) implies that every edge deletion creates an induced C_{14} . Finally, Proposition 4.2 gives an induced path on 14 vertices between the endpoints of every nonedge. Lemma 2.2(c) therefore implies that every nonedge addition creates an induced C_{14} . $\square$

5. COMPUTATIONAL VERIFICATION AND REPRODUCIBILITY

5.1. Verification layers. The released computations keep the following logically different tasks separate.

For G_{12} , the primary program constructs the graph from an embedded LCF shift sequence and exhaustively enumerates the 12-cycles and induced 12-vertex paths. Its essential gates use Python `assert`, so it must not be run with `python -O`. The separately written verifier instead reads the edge list and certificate and uses only the Python standard library. It rejects duplicate raw JSON object keys and duplicate normalized toggle keys, checks exact graph and certificate coverage, and uses explicit runtime checks rather than assertions. In addition to validating every stored witness, it implements a different cycle canonicalization, a direct induced-cycle search in the original graph, direct searches after all 273 edge deletions, and an exhaustive induced-path endpoint computation covering all 16,198 additions. It shares the certificate reduction with the primary program but imports none of its code.

For G_{14} , both Python verifiers begin from Equation (1), read all three released data representations, and validate all 378 stored toggle witnesses. Their JSON readers reject duplicate raw object keys, noncanonical pair keys, and normalization collisions. Neither program contains a correctness-critical `assert`; all gates remain active under `python -O`. The second program is a separately written, code-independent implementation using integer masks, meet-in-the-middle cycle assembly, a different rotation/reversal canonicalization, and a bit-mask induced-path search. The two programs implement the same mathematical reduction and share only the released data format.

TABLE 4. Roles of the computational checks.

Check	What it establishes
Certificate validation	Every stored order is a valid toggle witness, and the distinct canonical keys cover exactly all edges or all nonedges.
Independent recomputation	Cycles, chords, induced paths, and endpoint coverage are recomputed from the graph without using the stored witness selected for a toggle.
Exhaustive enumeration	The described finite cycle, path, or fixed-size subset search space is traversed completely.
Graph identity	Exact labelled edge-set equality or a separately executed graph-isomorphism test relates released representations.
File integrity	SHA-256 digests identify the released bytes; they do not prove any graph-theoretic statement.

5.2. The all-subsets check. The C++20 program for G_{14} supplies a third check of the negative condition. It reconstructs the graph from the Fano-plane definition and enumerates exactly

$$\binom{28}{14} = 40,116,600$$

subsets of 14 vertices in the original graph. On each subset it computes all induced degrees and, if every degree is 2, checks connectedness. A simple 14-vertex graph is an induced C_{14} exactly when it is connected and 2-regular. The program examines the full subset count and finds none. This C++ program checks only the absence of an induced C_{14} in the original graph; it does not check either toggle condition, the certificate, the edge file, or the label map.

5.3. Reruns, software, and file integrity. Fresh runs from clean extractions reproduced the mathematical output of both core C12 programs and both C14 Python programs. The C14 C++ source compiled with C++20 and strict warning flags, and both the optimized and AddressSanitizer/UndefinedBehaviorSanitizer runs examined all 40,116,600 subsets without a diagnostic. The

core Python programs require Python 3; all except the optional C12 graph-identity comparison use only the standard library. That optional comparison requires NetworkX.

The optional stored C12 identity log records the SHA-256 digest of an earlier full census file rather than the digest of the extracted LCF line now bundled. A fresh run against the manifest-checked bundled line nevertheless reproduces the graph-level conclusions: equal order and size, girth 7, diameter 8, and an isomorphism with 182 mapping entries. The isomorphism claim in [Section 3](#) relies on this fresh run, not on equality of the two digest lines.

Both `SHA256SUMS.txt` manifests pass for every listed payload file. These hashes detect accidental changes and identify the inputs included in the released bundles. They are not evidence that a program was executed and are not mathematical evidence for induced saturation. The README files give the interpreter and compiler commands needed to reproduce the computations.

6. CONCLUDING REMARKS

The same short certificate lemma accounts for both constructions: cycles with a prescribed unique chord control deletions, while induced paths with a prescribed endpoint pair control additions. The enumerations reveal further regularity not needed for the proofs. In G_{12} , unique-chord 12-cycles correspond one-to-one with edges. In the Coxeter graph, every edge occurs six times as a unique chord, and the induced-path multiplicity depends only on endpoint distance.

The examples do not address minimum order, nor do they by themselves give a general construction for longer even cycles. It remains natural to ask whether comparable certificates occur in other cubic symmetric graphs and whether automorphism-orbit arguments can compress the released witnesses. No claim of priority or minimality is made for either graph.

ACKNOWLEDGEMENTS

AI tools assisted in candidate generation and in drafting portions of the verification code and exposition. The claims recorded here are supported by the accompanying machine-readable certificates and exhaustive verification programs described in [Section 5](#). These computations are not formal proof-assistant developments, and some implement the same mathematical reduction. This manuscript is a preliminary computational record and has not been peer reviewed.

REFERENCES

1. Maria Axenovich and Mónika Csikós, *Induced saturation of graphs*, Discrete Mathematics **342** (2019), no. 4, 1195–1212, <https://doi.org/10.1016/j.disc.2019.01.006>.
2. Sarah Behrens, Catherine Erbes, Michael Santana, Derrek Yager, and Elyse Yeager, *Graphs with induced-saturation number zero*, The Electronic Journal of Combinatorics **23** (2016), no. 1, Paper No. P1.54, <https://doi.org/10.37236/5095>.
3. Marston Conder and Peter Dobcsányi, *Trivalent symmetric graphs on up to 768 vertices*, Journal of Combinatorial Mathematics and Combinatorial Computing **40** (2002), 41–63, <https://combinatorialpress.com/jcmcc-articles/volume-040/trivalent-symmetric-graphs-on-up-to-768-vertices/>.
4. Marston D. E. Conder and Primož Potočnik, *CAT(182,3) description*, Foster Census, GraphSym, Accessed 25 July 2026, https://fostercensus.graphsym.net/files/CAT-descriptions/CAT_182_3.html.
5. ———, *Edge-transitive cubic graphs: analysis, cataloguing and enumeration*, Journal of Algebra **685** (2026), 703–737, <https://doi.org/10.1016/j.jalgebra.2025.07.035>.
6. H. S. M. Coxeter, *Self-dual configurations and regular graphs*, Bulletin of the American Mathematical Society **56** (1950), no. 5, 413–455, <https://doi.org/10.1090/S0002-9904-1950-09407-5>.
7. Xinyue Fan, Sahab Hajebi, Sepehr Hajebi, and Sophie Spirkl, *Halfway to induced saturation for even cycles*, 2025, arXiv:2505.24100v2 [math.CO], <https://arxiv.org/abs/2505.24100>.
8. ———, *Asymmetric induced saturation*, 2026, arXiv:2606.24763v1 [math.CO], <https://arxiv.org/abs/2606.24763>.
9. Ararat Harutyunyan, Reza Naserasr, Mirko Petruševski, Riste Škrekovski, and Qiang Sun, *Mapping planar graphs into the Coxeter graph*, Discrete Mathematics **339** (2016), no. 2, 839–849, <https://doi.org/10.1016/j.disc.2015.10.007>.
10. Ryan R. Martin and Jason J. Smith, *Induced saturation number*, Discrete Mathematics **312** (2012), no. 21, 3096–3106, <https://doi.org/10.1016/j.disc.2012.06.015>.